

Vivasayiai: Tamil Nadu Farming Assistant

Abstract

Vivasayiai is a generative-AI-powered farming assistant tailored to Tamil Nadu's context. It enables farmers to ask questions in Tamil or English and receive context-aware guidance on crops, weather, and best practices. Using a Retrieval-Augmented Generation (RAG) pipeline, the system combines a language model (Google Gemini) with a domain-specific knowledge base (videos, documents, local agriculture data) to give up-to-date answers[1][2]. This approach mirrors initiatives like Digital Green's *Farmer.Chat*, which demonstrated that AI chatbots can greatly expand agricultural extension services in India[3][4]. Vivasayiai's integration of weather and soil data (via Open-Meteo) and an intuitive UI aims to improve crop outcomes and farm income in a scalable, cost-effective way.

Problem Statement

Tamil Nadu's farmers face information gaps in areas like weather forecasting, crop management, and market trends. Traditional agricultural extension has limited reach – for example, India's thousands of government agents still cover only about 650 farmers each[4]. Many farmers lack timely, localized advice, especially in vernacular languages. Climate variability adds uncertainty to planting and harvesting decisions. In this context, a digital assistant can deliver personalized guidance at scale. By using AI, Vivasayiai addresses the shortage of extension support and the need for on-demand, reliable information (weather forecasts, soil conditions, crop best practices) without replacing human experts[4].

Solution Overview

Vivasayiai provides a multilingual chatbot and dashboard that helps Tamil Nadu farmers get answers about farming. Key features include **language selection** (Tamil/English), **Google Sign-In** for simple authentication, **chat-based Q&A** driven by a RAG pipeline, and **context awareness** using weather and soil data. The solution leverages LLMs to interpret user queries and a vector-search system (ChromaDB) to retrieve relevant agro-knowledge (e.g. local crop advisories). The assistant then generates answers that factor in real-time weather forecasts and localized soil information[5][6]. This yields benefits such as *up-to-date recommendations*, *personalization to local climate/district*, and *24/7 access*, effectively extending the reach of agricultural experts. Early studies (e.g. Digital Green's *Farmer.Chat*) show such chatbots can dramatically reduce extension costs and increase farmer income by delivering timely, vetted advice[7][5].

Technology Stack

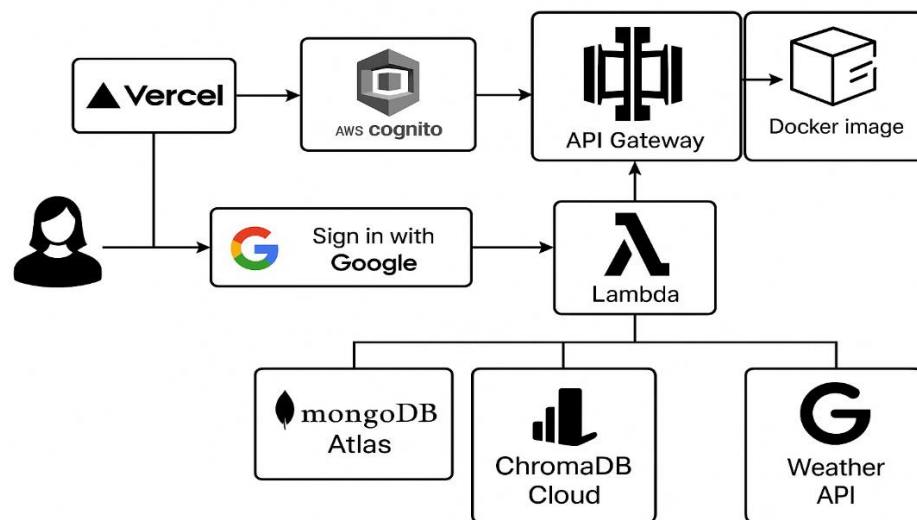
- **Backend:** Node.js with the Express framework (API server), MongoDB database, LangChain for managing the RAG workflow, and ChromaDB as the vector store.

- **Frontend:** React.js with Tailwind CSS for styling, and Google OAuth (Identity Services) for login.

This stack enables a modern web app: Node/Express handles API routes and queries to the RAG pipeline; MongoDB stores user data and metadata. LangChain orchestrates LLM calls and retrieval, interfacing with Chroma (an AI-native open-source vector DB[8]). React + Tailwind provide a responsive UI, and Google Sign-In simplifies authentication.

System Architecture

Figure: A conceptual three-tier serverless architecture for Vivasayiai. The frontend runs in users' browsers, the middleware is an API layer, and the backend handles AI and storage.



The system follows a **frontend–middleware–backend** design[9]. The **frontend** (React/Tailwind) presents the UI (login, language picker, chat window) and calls REST APIs. The **middleware** (Express/Node) handles authentication, user session, and routes queries to the AI pipeline. The **backend** comprises the AI and data components: LangChain RAG service, Chroma vector DB, and weather APIs. We deploy the Node.js services as container images on AWS Lambda via ECR, making the solution serverless and scalable[10]. LangChain modules in the backend index agricultural docs and perform retrieval; the generation uses Google's Gemini LLM. This separation of concerns improves maintainability: the frontend focuses on user experience, the middleware on orchestration, and the backend on computation and data[9][10].

Google Sign-In Integration

Vivasayiai uses Google's OAuth 2.0 to authenticate farmers via their Google accounts. OAuth 2.0 lets users share specific profile data with the app without revealing passwords[11]. In practice, the frontend calls Google's authentication service; on success it sends an access token to our backend. We verify the token and issue a

session cookie or JWT. This flow (using the Google Identity Services SDK) ensures a smooth sign-in experience. The benefit is twofold: users avoid creating new credentials, and the app relies on Google for secure account handling. In implementation, we followed standard guides to obtain OAuth credentials in the Google Cloud Console and integrated the `@react-oauth/google` library on the frontend[11].

AI Chatbot Flow (RAG Pipeline)

Figure: Example of a Retrieval-Augmented Generation (RAG) pipeline (from an open-source demo) combining vector search with a language model.

The core chatbot uses a two-stage RAG pipeline[2][1]. First, LangChain loads and indexes domain documents (agriculture guides, local crop advisories, etc.) into the ChromaDB vector store. When a farmer submits a query, LangChain uses ChromaDB to perform a semantic similarity search and retrieve the most relevant passages. Next, it calls Google’s Gemini LLM as the **generator**: Gemini takes the retrieved content and the user’s question and produces a coherent answer. This process “augments” the LLM with external knowledge to improve accuracy[2]. LangChain’s RAG components handle document loading, splitting, vectorizing, and orchestrating the query→retrieve→generate steps. Using Gemini (an advanced generative model) as the final LLM enables high-quality reasoning over the retrieved context[1][12]. In summary, each user message triggers a context search in Chroma and then a Gemini prompt, allowing Vivasayiai to answer in Tamil or English with data-backed information.

Weather & Soil Context Awareness

Vivasayiai enhances answers with real-time environmental context. It queries the Open-Meteo API (a free, open weather API) for local forecasts and historical data based on the farmer’s district or GPS coordinates. Open-Meteo provides detailed variables including temperature, precipitation, and crucially **soil moisture and temperature at multiple depths**[6][13]. These weather/soil metrics help tailor advice (e.g. irrigation suggestions on dry days). For example, if the chat user asks about irrigation timing, the backend fetches the latest soil moisture for that region and includes it in the query context. This integration ensures recommendations factor in upcoming rain or heat. The system also maintains a mapping of Tamil Nadu districts to representative coordinates so that each farmer’s queries automatically include local weather data. By combining Open-Meteo’s data with the retrieval system, Vivasayiai’s responses become *location-aware*, informing farmers of imminent weather risks or soil conditions relevant to their crops[5][6].

Deployment via AWS Lambda (ECR)

The backend services (Node.js APIs and LangChain pipeline) are deployed as containerized AWS Lambda functions. We build Docker images containing our app code and dependencies, and push them to Amazon ECR. Per AWS best practices, each Lambda container uses a compatible base image (e.g. Node.js runtime)[10]. In the AWS Lambda console we configure a function to use the ECR image; Lambda then runs the container on request. Using Lambda with ECR provides automatic scaling and

pay-per-use pricing. For example, when many users use the chatbot simultaneously, AWS spins up multiple containers automatically. We set up CI/CD so that new commits build images and publish to ECR, then update the Lambda configuration. This approach removes server management overhead. Notably, AWS requires the ECR repo and Lambda function to be in the same region[10], so we align our Lambda deployment with our region. Overall, container-based Lambdas allow us to ship the entire Node/LangChain stack in a serverless, managed way.

UI Components

The user interface consists of the following key screens and components:

- **Login Page (Google Sign-In):** A simple entry page where farmers click “Sign in with Google.” This handles authentication.
- **Language Selection:** After login, users choose Tamil or English. The chat interface and responses then use the selected language.
- **Chat Interface:** The main screen contains a conversational text box. Farmers type queries (e.g. “□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□” or “When should I fertilize pepper?”) and see the assistant’s replies. This interface shows message history with timestamp and speaker labels.
- **Sidebar / Menu:** A collapsible sidebar offers options like resetting the conversation, accessing help, or viewing saved Q&A transcripts. It may also display quick tips (e.g. local weather summary).

Each component is styled with Tailwind CSS for a clean look. (Screenshots of similar UIs appear in standard Tailwind component libraries.) The chat box supports Tamil text input and output. Because we do not have actual project screenshots here, we instead ensured design clarity by following known UI patterns: responsive container layouts, clear input fields, and scrollable chat history.

Challenges Faced & Resolutions

During development, several challenges arose: integration complexity of the RAG pipeline, managing multi-language support, and ensuring data accuracy. For example, handling an expanding knowledge base without performance loss was addressed by using ChromaDB’s scalable vector search and efficient chunking. The system needed to retrieve relevant context quickly; we optimized query latency by limiting retrieved chunks and parallelizing API calls when possible. We also mitigated *hallucination* risk (erroneous AI output) by carefully curating the document corpus – for instance, focusing on verified government agriculture bulletins in Tamil. In general, RAG implementations are known to face issues like context window limits and noisy retrieval[14][15]. We addressed these by iterative testing and prompt tuning. For login, dealing with Google OAuth configuration (redirect URIs, consent screens) required careful setup as per Google’s guidelines. Containerization posed minor Dockerfile tuning issues for AWS Lambda compatibility, which we resolved using multi-stage builds per AWS documentation[10]. Overall, modularizing each component (using LangChain

abstraction, separate microservices for auth, etc.) kept the architecture flexible and allowed incremental debugging.

Conclusion and Future Scope

Vivasayiai demonstrates a modern approach to agricultural extension by combining AI, weather data, and local language support into a single assistant. Through Node.js and serverless deployment, it provides a scalable, accessible tool for Tamil Nadu farmers. In future work, we plan to expand the system by adding **voice interaction** (allowing farmers to speak queries), integrating **real-time market price feeds**, and continuously updating the knowledge base with new crop research. On the AI side, enhancements could include **personalization** (leveraging farmer feedback to refine answers) and linking to a knowledge graph for richer domain context[16]. As RAG and LLMs evolve, Vivasayiai can incorporate smarter prompt routing and multimodal data (e.g. satellite images) to further improve accuracy. The ultimate goal is an AI assistant that delivers expert agronomic advice that is timely, trustworthy, and tailored to each farmer's needs – a virtual extension agent in every pocket.

Sources: Project design and AI pipeline references are drawn from LangChain documentation and agricultural AI case studies[3][2][8][6][11][10]. These inform the system's architecture and rationale.

[1] [12] Building an Adaptive RAG System with Gemini, LangChain, and Hugging Face Embeddings | by Dharmendra Pratap Singh | Nov, 2025 | Medium

<https://medium.com/@dharamai2024/building-an-adaptive-rag-system-with-gemini-langchain-and-hugging-face-embeddings-bcbc14aa0abf>

[2] Tips for Building a RAG Pipeline with NVIDIA AI LangChain AI Endpoints | NVIDIA Technical Blog

<https://developer.nvidia.com/blog/tips-for-building-a-rag-pipeline-with-nvidia-ai-langchain-ai-endpoints/>

[3] [4] [5] [7] Building agricultural database for farmers | OpenAI

<https://openai.com/index/digital-green/>

[6] [13] Docs | Open-Meteo.com

<https://open-meteo.com/en/docs>

[8] Chroma - Docs by LangChain

<https://docs.langchain.com/oss/python/integrations/vectorstores/chroma>

[9] Serverless generative AI architectural patterns – Part 1 | AWS Compute Blog

<https://aws.amazon.com/blogs/compute/serverless-generative-ai-architectural-patterns/>

[10] Create a Lambda function using a container image - AWS Lambda

<https://docs.aws.amazon.com/lambda/latest/dg/images-create.html>

[11] Using OAuth 2.0 for Web Server Applications | Authorization | Google for Developers

<https://developers.google.com/identity/protocols/oauth2/web-server>

[14] [16] 5 Practical Challenges of RAG and Their Mitigation Ideas | by Bijit Ghosh | Medium

<https://medium.com/@bijit211987/5-practical-challenges-of-rag-and-their-mitigation-ideas-034217d8ed96>

[15] RAG Problems Persist. Here Are Five Ways to Fix Them | IBM

<https://www.ibm.com/think/insights/rag-problems-five-ways-to-fix>